Comparing Traditional and Streaming Machine Learning Methods for Soccer Pass Detection

Laia Ferrer Moser

Politecnico di Milano
Streaming Data Analytics
Prof. Emanuele Della Valle

Abstract. This part of the thesis explores the use of Esper, a Java-based software designed for Complex Event Processing (CEP), as a solution to the DEBS 2013 Grand Challenge. Initially, we developed a baseline implementation that processed all incoming data collectively, without distinguishing between individual sensors or players. Our evaluation demonstrates the effectiveness of the proposed system, achieving low-latency processing suitable for real-time environments.

Keywords: Complex Event Processing, Real-Time Event Processing, SensorEvent, BallEvent, DEBS 2013 Grand Challenge, Esper

1 Introduction

Complex Event Processing (CEP) systems are designed to analyze large volumes of event data in real time, making them a key technology for identifying situations of interest in high-throughput data streams [1]. CEP has a wide range of applications across various domains — including sports, where it enables instantaneous reactions to ongoing events. Numerous studies have highlighted the advantages of CEP [2, 3, 4], demonstrating its potential to improve decision-making, responsiveness, and overall system efficiency.

1.1 DEBS 2013 Grand Challenge

The ACM DEBS 2013 Grand Challenge is the third in a series of challenges that seek to provide a common ground and evaluation criteria for a competition aimed at research and industrial event-based systems. The goal of the Grand Challenge competition is to implement a solution to a problem provided by the Grand Challenge organizers. The DEBS Grand Challenge series presents problems that are relevant to the industry at large. DEBS Grand Challenge problems allow for evaluation of event-based systems using real-life data and queries. 2013 DEBS Grand Challenge seeks to demonstrate the applicability of event-based systems to provide real-time complex analytics over high-velocity sensor data along with the example of analyzing a soccer game. [5] In this scenario, sensors are attached to players, referees, and the ball to collect positional and motion

data at each point in time. Each player is equipped with two sensors—one on each leg—while the goalkeeper wears two additional sensors, one on each hand. Using the continuous data streams from all these sensors, the objective is to design and implement an event processing system capable of ingesting the full set of sensor data and computing the results for the defined queries. It is important to note that, in order to meet real-time requirements, the system must adhere to strict constraints on both latency and throughput.

The dataset used in this work was collected during a real football match recorded at the Nuremberg Stadium. The game was played on a half-sized field and divided into two halves of 30 minutes each. The dataset comprises raw sensor measurements, sampled at a frequency of 200 Hz for each player sensor and 2000 Hz for the ball, providing almost 16000 events/second in total.

The structure of the stream is represented by a 13-tuple:

(sid, ts, x, y, z, —v—, —a—, vx, vy, vz, ax, ay, az)

where sid is a unique sensor identifier, ts is the timestamp in picoseconds, x, y, z describe the position of the sensor in mm, —v— (in µm/s), vx, vy, vz represent a vector for the speed and —a— (in $\mu m/s^2$), ax, ay, az describe the absolute acceleration. In addition to the raw sensor data, a metadata file is included, which specifies team distributions and player names.

The paper is structured as follows: Section 1 introduces the DEBS 2013 Grand Challenge and outlines the four analytical queries. Section 2 describes the overall system architecture and discusses key implementation aspects. Section 3 presents the detailed solutions developed for each query. In Section 4, we share our experience building and executing the system, along with the obtained performance results. Finally, the paper concludes with a discussion of related work and potential directions for future research.

1.2 Queries

Four analytical queries are defined, and their results must be generated at the same frequency at which events are received, unless otherwise specified.

Query 1: Running Analysis Query 1 aims to calculate the running performance of each player participating in the game. The system distinguishes between six predefined running intensity levels: stop, trot, low, medium, high, and sprint. This query has two primary objectives: 1. To compute the duration and distance of each individual run segment. 2. To aggregate these computations into cumulative running statistics per player. The first part of the query produces output in the form: (tsStart, tsStop, playerId, intensity, distance, speed) The second part generates aggregated statistics with the following structure: (ts, playerId, standingTime, standingDistance, lowTime, lowDistance, mediumTime, mediumDistance, highTime, highDistance, sprintTime, sprintDistance). The query is computed over four different time windows (1, 5, 20 minutes, and the hole game).

Query 2: Ball Posession Query 2 aims to compute ball possession statistics at both the player and team levels. A player is considered to be in possession of the ball when two conditions are met: The ball is within close proximity to the player — specifically, less than 1 meter between the player and the ball sensor. The ball's acceleration exceeds 55 m/s^2. Once possession is established, the ball is considered to remain under that player's control until one of the following occurs: Another player gains possession based on the same criteria, the game is stopped, or the ball goes out of bounds. The result should have the following structure (ts, playerId, time, hits) for players and (ts, teamId, time, percentage) for teams. The last one is also computed over four time windows (1, 5, 20 minutes, and the hole game).

Query 3: Heat Map Query 3 aims to calculate statistics that reflect how much time each player spends in different regions of the playing field. To achieve this, the field is divided into a grid defined by X rows and Y columns. Four grid resolutions are considered: 8×13 (104 cells), 16×25 (400 cells), 32×50 (1600 cells) ,64×100 (6400 cells). For each resolution, the system calculates the percentage of time a player spends within each cell. These statistics are computed over four time windows: 1 minute, 5 minutes, 10 minutes, and the full duration of the game. As a result, the query produces 16 output streams, each updated every second. Each output record has the following structure: (ts, playerId, cellX1, cellY1, cellX2, cellY2, percentage) where the cell is defined by its lower-left corner (x1, y1) and upper-right corner (x2, y2).

Query 4: Shot on Goal Query 4 is designed to identify shots that are aimed at scoring a goal—specifically, those that either hit or narrowly miss the opposing team's goal. A shot is considered a shot on goal if it satisfies the same conditions defined in Query 2 for shot detection, with the additional requirement that the projected trajectory of the ball crosses the opponent's goal line within 1.5 seconds of being taken. While the shot is in progress, the query outputs a continuous result stream at the same rate as the SensorEvent. Each result has the following structure: (ts, playerId, x, y, z, —v—, vx, vy, vz, —a—, ax, ay, az)

2 Architecture

In this section, we present the architecture developed to address the problem. The system begins by reading the raw sensor data from the provided file. Each line in the file corresponds to a single sensor reading and follows the structured format previously described. Before event streaming starts, a metadata file is parsed to associate each sensor with the corresponding player and their team. Once the metadata has been processed, the sensor data is loaded into memory using the streamSensorData() method, which constructs a list of SensorEvent objects.

After all events have been parsed and stored, the method sendAllSensorEventsSingleThread() is called to stream the events to the Esper CEP engine. During

this phase, events are classified and dispatched based on their type. Events with sensor IDs equal to 4, 8, 10, or 12—corresponding to the ball—are converted into BallEvent objects and sent directly to Esper. All other events are treated as SensorEvent instances and sent accordingly.

A specific mechanism was implemented to support Query 1: if a new SensorEvent has a different movement intensity than the previous event from the same player, the event is sent twice. This approach ensures that the current context in Query 1 is properly closed, while simultaneously opening a new context corresponding to the updated intensity level.

For the Complex Event Processing (CEP) engine, Esper was selected due to its seamless integration with Java-based projects and its ability to handle high-throughput, real-time event streams efficiently. One of Esper's key strengths is its use of a high-level declarative language known as Event Processing Language (EPL), which closely resembles SQL and simplifies the task of filtering, correlating, and analyzing event patterns. In this project, Esper version 7.1.0 was used to implement and execute the analytical queries defined by the DEBS 2013 Grand Challenge.

3 Queries Solution

We now focus on the solution I have created for the different queries proposed by the challenge.

3.1 Query 1

Query 1 monitors each player's running performance by tracking the total distance covered and categorizing it based on predefined intensity levels. To achieve this, a context is created for every SensorEvent received from a player and remains active until the player's movement intensity changes." As previously said, the events that terminate the context will be sent twice in order to finish one context and also initiate another one. Within each context, the distance covered is estimated by calculating the average velocity and multiplying it by the duration for which the player maintained that specific intensity. Once a context ends, the corresponding time and distance are added to the player's aggregate running statistics.

To efficiently manage and store this data, a Map structure is used, where the playerId serves as the key and the value is an instance of the RunningStatistics class. This class encapsulates all relevant attributes, such as cumulative time and distance for each intensity level, enabling quick retrieval and updates throughout the game.

3.2 Query 2

Query 2 is responsible for detecting ball possession at both the player and team levels. To facilitate this, two event windows are maintained: one stores the most

recent BallEvent that has not yet been associated with a hit, and the other stores the latest position of each player's sensor. These windows are continuously updated, always holding only the most recent relevant event(s). Using these two windows, an EPL query is defined to detect possession events based on the following conditions: 1.The distance between the player's sensor and the ball is less than 1 meter. 2.The ball's acceleration exceeds 55 m/s^2. 3.The timestamp of the player's event is greater than the timestamp of the ball's event. When all three conditions are satisfied, a ShotEvent is generated, indicating that a player has taken a shot. This event is also used in Query 4 to determine whether the shot qualifies as a shot on goal. To prevent the same BallEvent from being associated with multiple hits, it is removed from the lastBallEvent window immediately after the ShotEvent is created. Following the generation of a ShotEvent, a secondary query processes consecutive possession events to determine the duration for which the ball remained under a player's control. To keep track of this, a Map structure is used, where each entry stores a player's current possession state, including cumulative possession time and number of hits. For calculating team-level ball possession, additional EPL queries are used. These queries compute the total number of possession events associated with each team (TeamA or TeamB) over a specific time window. By summing the durations of all possession instances per team and comparing them against the total time covered by the window, the system calculates the percentage of time each team maintained possession of the ball.

3.3 Query 3

Query 3 is designed to generate a heat map that reflects the time each player spends in different regions of the playing field. While the challenge specification defines multiple grid resolutions—varying in the number of rows and columns—and multiple time windows, computing and storing heat maps for every possible configuration would lead to excessive memory consumption. To address this, the system was designed to compute the heat map using only the smallest cell size grid resolution. This approach allows for greater memory efficiency and flexibility, as the other resolutions can be derived later by aggregating adjacent cells and recomputing the corresponding percentages.

The implementation involves a query that stores the last two SensorEvents for each player. Although a player may have up to four sensors, the system simplifies position tracking by assuming that the player's current position corresponds to the most recent event received for that player.

For each pair of consecutive events from the same player, the system computes the time interval between them. It then updates the total time the player spent within the corresponding grid cell during that interval. Simultaneously, it maintains the total time accumulated over each time window (e.g., 1, 5, 10 minutes, or the entire game). Once these values are collected, the percentage of time spent in each cell for a given time window can be calculated.

3.4 Query 4

Query 4 is responsible for identifying shots on goal, distinguishing them from general shots detected in Query 2. It operates by identifying ShotEvent instances, based on a set of predefined conditions (e.g., player proximity to the ball, acceleration threshold, and correct timestamp order). Once a shot has been detected, Query 4 determines whether the shot is intended to score a goal which we check with whether it is directed toward the opponent's goal and would either hit or narrowly miss it. To perform this check, the system analyzes the trajectory of the ball based on its current position and velocity in the x and y directions. The ball's projected position is calculated 1.5 seconds into the future using a linear approximation, assuming no acceleration or gravity. Specifically, the projected position is computed as: xFinal = xCurrent + vx * t // yFinal = yCurrent + vy * t where t = 1.5 seconds. The system then checks whether this projected position falls within an extended goal area, defined as the official goal dimensions plus an additional 2.5 meters of margin on each side. This margin accounts for near misses that still qualify as intentional goal attempts. If the projected trajectory meets the criteria, a GoalShotEvent is emitted. This event initializes a context that remains active for the duration of the shot. The context allows the system to continuously output the relevant data stream associated with the shot. The context is terminated under one of the following conditions: 1. The ball leaves the boundaries of the field, 2. The game is stopped, 3. The ball's trajectory changes such that it no longer points toward the goal. This approach enables continuous monitoring and analysis of goal-oriented shots in real time, while maintaining a balance between computational simplicity and practical accuracy.

4 Results

To comply with the required output formats for the DEBS Grand Challenge, we created four separate output files—one for each query—where the corresponding result streams are printed. Since the amount of data that needs to be saved is huge, in order to test the throughput and latency of the program, we just send around 7 million tuples, that after applying data cleaning and filtering steps, the number of valid events processed was reduced to around 4 million. Based on this volume, we conducted performance analysis and obtained the system's event throughput, which is discussed in the following paragraph.

At its peak performance, the system is capable of processing approximately 31,362 events per second. Executing the simplified dataset—consisting of nearly 7 million tuples—took approximately 130 seconds. This demonstrates a significantly higher throughput than what would be required in a real-time scenario, where data is expected to arrive at specific frequencies. In particular, the challenge specifies a rate of 200 Hz for player sensors and 2000 Hz for the ball sensor.

To address this, future work focused on modifying the system to respect these frequency constraints will be done. By introducing timing logic and adjusting the event dispatch mechanism to account for the timestamp of each event, the system will be updated to emulate real-time streaming behavior.

5 Conclusions

This work presents a complete solution to the DEBS 2013 Grand Challenge, leveraging stream processing techniques and the Esper Complex Event Processing (CEP) engine. The proposed approach focuses on query parallelization and data stream flow control to efficiently process high-frequency sensor data in a centralized architecture.

While the centralized design proved effective, future work could explore a more decentralized architecture, where data is partitioned by player or even by individual sensor. In such a setup, queries would be directly fed only with the relevant subset of data, rather than the entire stream. This approach would enable a detailed comparison between centralized and decentralized processing strategies in terms of performance, scalability, and efficiency, and could potentially lead to more optimized real-time event processing pipelines.

References

1. C. Mutschler, et al. "DEBS 2013 Grand Challenge Soccer Monitoring Dataset." https://cmutschler.de/datasets/debs-2013-grand-challenge-soccer-monitoring
2. Simsek, M.U., Yildirim Okay, F.Ozdemir. "A deep learning-based CEP rule extraction framework for IoT data" Springer Nature, 2021. https://link.springer.com/article/10.1007/s11227-020-03603-5
3. David B. Robins "Complex Event Processing" University of Washington, 2010. https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=26f545cfe31efaee93e41291109f3b96e5d70921
4. G. Cugola, A. Margara "Processing Flows of Information: From Data Stream to Complex Event Processing" Politecnico di Milano, 2012. https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=26f545cfe31efaee93e41291109f3b96e5d70921
5. Alejandro Grez, Cristian Riveros, Martín Ugarte "A Formal Framework for Complex Event Processing" ICDT 2019. https://drops.dagstuhl.de/storage/00lipics/lipics-vol127-icdt2019/LIPIcs.ICDT.2019.5/LIPIcs.ICDT.2019.5.pdf